2026학년도 데이터베이스

개인 발표 기술 문서

MySQL을 활용한 대학생 지출 관리

지출 기록부터 월별·항목별 조회까지

구분	내용
과목명	데이터베이스
학과	디지털보안학과
학번	2025011890
이름	이승민
분반	_____ (분반이 있는 경우 기입)
제출일	2026년 ____월 ____일
제출 기한	2026년 11월 25일(수)

표지와 본문을 하나의 파일로 구성하였다.

1. 주제와 실습 목표

이 문서는 MySQL에 대학생의 가상 지출 기록을 저장하고, 원하는 조건에 맞춰 조회·계산하는 과정을 정리한 개인 발표 기술 문서이다. 개인발표계획서에서 정한 주제와 범위를 실제 SQL 파일과 실행 결과로 연결하였다.

생활 속 질문을 SQL로 바꾸기

궁금한 점	실습에서 확인할 내용
10월에 얼마를 썼는가?	10월 기록만 골라 건수와 금액 합계 계산
어떤 항목에 많이 썼는가?	식비·쇼핑 등 같은 항목끼리 묶어 합계 비교
가장 큰 지출은 무엇인가?	금액이 큰 순서로 정렬해 상위 5건 확인
다른 달 기록이 섞이지 않았는가?	9월·11월 기록을 넣고 날짜 조건 검증

처음 보는 용어를 쉽게 이해하기

용어	이 문서에서의 뜻
데이터베이스	관련된 정보를 일정한 규칙에 맞춰 모아 둔 저장 공간
MySQL	데이터베이스에 정보를 저장하고 관리하는 프로그램
SQL	저장·조회·수정 같은 작업을 요청하는 명령어
테이블(표)	같은 형식의 기록을 행과 열로 정리한 구조
행 / 열	행은 지출 한 건, 열은 날짜·금액 같은 정보의 종류
조회 / 집계	조회는 필요한 기록 찾기, 집계는 합계·건수 계산하기

문서 구성

2장 데이터 설계 → 3장 표 만들기와 입력 → 4장 월별 조회 → 5장 항목별 합계 → 6장 정렬과 월별 비교 → 7장 수정·삭제 연습 → 8장 검증과 보안 → 9장 실행 안내와 결론 → 10장 참고 자료

모든 소비 내역과 사용처는 실습용 가상 정보이다. 실제 이승민 학생의 소비 내역이나 조사 결과를 의미하지 않는다.

2. 어떤 정보를 저장할 것인가?

한 번 지출할 때마다 한 행을 추가한다. 이번 실습은 한 사람의 지출 기록을 다루므로 하나의 표만 사용한다. 로그인이나 웹 화면은 실습 범위에 포함하지 않는다.

열 이름	쉬운 설명	저장 형식 / 예시
expense_id	지출 한 건을 구별하는 번호	자동으로 증가하는 정수 / 3
spent_on	돈을 사용한 날짜	DATE: 날짜 / 2026-10-01
category	지출 항목	VARCHAR(20): 짧은 글 / 식비
place	돈을 사용한 곳	VARCHAR(60): 짧은 글 / 가상 학생식당
amount	사용한 금액	INT UNSIGNED: 0 이상 정수 / 6500

금액의 단위는 원이며, 소수점이 없는 금액만 다룬다. 이 설계는 일반 지출용이므로 환불처럼 음수 금액을 넣는 상황은 다루지 않는다. NOT NULL은 빈값(NULL)을 허용하지 않는 설정이지만, 글자 수가 0인 문자열까지 자동으로 막는 것은 아니다.

가상 기록의 구성

기간	건수	넣은 이유
2026년 9월 30일	2건	10월 이전 기록이 제외되는지 확인
2026년 10월 1~31일	24건	주요 조회·계산 대상
2026년 11월 1일	2건	다음 달 기록이 제외되는지 확인
합계	28건	전체 데이터

데이터 일부 예시

날짜	항목	사용처	금액(원)
2026-10-01	식비	가상 학생식당	6,500
2026-10-01	교통비	가상 버스	1,500
2026-10-02	카페	가상 카페	4,500
2026-10-03	쇼핑	가상 문구점	23,000

전체 28건은 함께 제공한 「가상_지출기록.csv」와 01번 SQL 파일에 수록하였다. CSV는 내용을 확인하기 위한 자료이며, SQL 실행 시 별도로 가져올 필요가 없다.

3. 표 만들기와 지출 기록 입력

먼저 정보를 담을 데이터베이스와 표를 만든다. 아래 데이터베이스 이름은 이 과제에서만 사용하는 연습용 이름이다. 같은 이름이 이미 있다면 기존 내용을 지우지 말고 다른 이름을 사용한다.

```
CREATE DATABASE student_expense_demo_2025011890
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE student_expense_demo_2025011890;

CREATE TABLE expenses (
  expense_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  spent_on DATE NOT NULL,
  category VARCHAR(20) NOT NULL,
  place VARCHAR(60) NOT NULL,
  amount INT UNSIGNED NOT NULL
) ENGINE=InnoDB;
```

명령 또는 설정	쉽게 풀어 쓴 역할
CREATE / USE	저장 공간이나 표를 만들기 / 사용할 저장 공간 선택
utf8mb4	한글 등 여러 문자를 저장할 수 있게 문자 형식 지정
PRIMARY KEY	각 행을 구별하는 대표 번호로 지정
AUTO_INCREMENT	번호를 하나씩 자동으로 부여
ENGINE=InnoDB	뒤의 선택 실습에서 변경 되돌리기를 지원하는 저장 방식 선택

한 건을 입력하는 모양

```
INSERT INTO expenses (spent_on, category, place, amount)
VALUES ('2026-10-01', '식비', '가상 학생식당', 6500);
```

INSERT INTO 뒤에는 입력할 표와 열을 쓰고, VALUES 뒤에는 실제 값을 쓴다. 날짜와 글자는 작은따옴표로 감싸며, 금액은 6500처럼 숫자로 입력한다. 자동 번호인 expense_id는 생략한다.

위 INSERT는 문법 설명용이다. 01번 SQL 파일에는 28건 전체 입력 명령이 있으므로, 그 파일을 실행한 뒤 위 예시를 추가 실행하면 기록이 한 건 더 생긴다. 예시를 중복 실행하지 않는다.

4. 10월 기록과 총지출 조회

4-1. 날짜 조건으로 필요한 기록만 찾기

```
SELECT expense_id, spent_on, category, place, amount
FROM expenses
WHERE spent_on >= '2026-10-01' AND spent_on < '2026-11-01'
ORDER BY spent_on, expense_id;
```

SELECT는 보고 싶은 열, FROM은 가져올 표를 정한다. WHERE는 조건에 맞는 행만 고른다. 여기서는 10월 1일 이상이면서 11월 1일 미만인 날짜를 선택하여 10월 31일까지 포함한다. ORDER BY는 날짜 순서로 결과를 정렬하고, 같은 날짜이면 지출 번호 순서로 보여 준다.

조회 결과 일부

번호	날짜	항목	사용처	금액(원)
3	2026-10-01	식비	가상 학생식당	6,500
4	2026-10-01	교통비	가상 버스	1,500
5	2026-10-02	카페	가상 카페	4,500
…	…	…	…	…
26	2026-10-31	교통비	가상 버스	1,500

전체 결과는 24건이다. 위 표는 처음 3건과 마지막 1건만 보여 준다.

4-2. 건수와 합계 구하기

```
SELECT COUNT(*) AS expense_count, SUM(amount) AS total_amount
FROM expenses
WHERE spent_on >= '2026-10-01' AND spent_on < '2026-11-01';
```

10월 지출 건수	10월 총지출(원)
24	195,700

출처: 제공 SQL을 MySQL 8.0.46에서 실행한 결과. 표의 제목은 이해하기 쉽게 바꾸었다.

COUNT(*)는 선택된 행의 수를 세고, SUM(amount)는 금액을 모두 더한다. AS 뒤의 이름은 결과 열에 붙이는 별명이다. 이 결과는 실습용 가상 기록의 합계이며, 대학생의 일반적인 월평균 지출을 뜻하지 않는다.

해당 기간에 기록이 한 건도 없다면 COUNT(*)는 0, SUM(amount)는 NULL이 된다. 실제 활용에서는 빈 기간에 0을 표시하도록 추가 처리할 수 있다.

5. 항목별로 묶어 합계 비교하기

```
SELECT category, SUM(amount) AS category_total
FROM expenses
WHERE spent_on >= '2026-10-01' AND spent_on < '2026-11-01'
GROUP BY category
ORDER BY category_total DESC, category;
```

GROUP BY category는 같은 항목을 가진 행을 한 묶음으로 만든다. 각 묶음의 금액을 SUM으로 더하고, ORDER BY의 DESC를 사용하여 큰 합계부터 보여 준다.

항목	10월 합계(원)
식비	71,000
쇼핑	56,000
문화생활	42,000
카페	19,000
교통비	7,700

출처: 제공 SQL을 MySQL 8.0.46에서 실행한 결과. 표의 제목은 이해하기 쉽게 바꾸었다.

가상 기록에서는 식비가 71,000원으로 가장 크다. 모든 항목의 합계를 더하면 195,700원으로, 앞에서 구한 10월 총지출과 같다. 항목을 '식비'와 '식사'처럼 다르게 입력하면 다른 묶음으로 계산되므로 같은 표현을 쓰는 것이 중요하다.

합계가 5만 원 이상인 항목만 보기

```
SELECT category, SUM(amount) AS category_total
FROM expenses
WHERE spent_on >= '2026-10-01' AND spent_on < '2026-11-01'
GROUP BY category
HAVING SUM(amount) >= 50000
ORDER BY category_total DESC, category;
```

WHERE는 묶기 전의 개별 기록을 고르고, HAVING은 묶어서 계산한 결과에 조건을 건다. 이 예시는 10월 기록을 먼저 고른 뒤, 합계가 50,000원 이상인 식비(71,000원)와 쇼핑(56,000원)만 보여 준다. 50,000원은 기능 설명을 위해 정한 기준이며 예산이나 과소비 판단 기준은 아니다.

6. 큰 지출과 월별 합계 확인

6-1. 금액이 큰 지출 5건

```
SELECT expense_id, spent_on, category, place, amount
FROM expenses
WHERE spent_on >= '2026-10-01' AND spent_on < '2026-11-01'
ORDER BY amount DESC, expense_id ASC
LIMIT 5;
```

번호	사용처	금액(원)
7	가상 문구점	23,000
24	가상 공연장	20,000
13	가상 서점	18,000
19	가상 생활용품점	15,000
8	가상 영화관	12,000

LIMIT 5는 결과를 5건까지만 보여 준다. 금액이 같으면 지출 번호가 작은 기록을 먼저 선택하여 실행할 때마다 순서가 일정하도록 했다. 12,000원 지출은 2건이지만, 이 조건에서는 번호 8번이 먼저 선택된다.

6-2. 월별로 묶기

```
SELECT DATE_FORMAT(spent_on, '%Y-%m') AS expense_month,
       COUNT(*) AS expense_count, SUM(amount) AS total_amount
FROM expenses
GROUP BY DATE_FORMAT(spent_on, '%Y-%m')
ORDER BY expense_month;
```

월	건수	합계(원)
2026-09	2	7,500
2026-10	24	195,700
2026-11	2	11,000

출처: 제공 SQL을 MySQL 8.0.46에서 실행한 결과. 표의 제목은 이해하기 쉽게 바꾸었다.

DATE_FORMAT(spent_on, '%Y-%m')은 날짜에서 연도와 월을 ‘2026-10’처럼 표시한다. 이 값을 기준으로 묶으면 여러 달의 합계를 한 번에 볼 수 있다. 다만 9월과 11월은 날짜 조건 검증을 위한 2건씩만 있으므로, 이 결과로 월별 소비가 늘거나 줄었다고 판단할 수는 없다.

7. 선택 실습: 잘못 입력한 기록 수정·삭제

핵심 발표는 조회와 합계 계산이다. 다음은 관련 기본 명령을 추가로 익히기 위한 선택 실습이다. 원래 28건을 건드리지 않도록 연습용 행을 따로 만들고, 같은 연결에서 아래 명령을 순서대로 실행한다.

```
START TRANSACTION;
INSERT INTO expenses
  (expense_id, spent_on, category, place, amount)
VALUES (900001, '2026-10-15', '카페', '수정삭제 연습용', 4500);

UPDATE expenses SET amount = 4000
WHERE expense_id = 900001;
SELECT expense_id, place, amount FROM expenses
WHERE expense_id = 900001;

DELETE FROM expenses WHERE expense_id = 900001;
SELECT COUNT(*) AS practice_rows_remaining
FROM expenses WHERE expense_id = 900001;
ROLLBACK;
```

순서	실제 확인 결과
연습 행 입력	번호 900001의 금액 4,500원
UPDATE 실행 후	같은 행의 금액 4,000원
DELETE 실행 후	번호 900001인 행의 수 0건
ROLLBACK 실행 후	원래 기록 28건, 전체 합계 214,200원 유지

UPDATE는 기존 값을 바꾸고, DELETE는 행을 지운다. 두 명령에서 WHERE 조건을 빠뜨리면 여러 기록을 한꺼번에 바꾸거나 지울 수 있으므로, 먼저 같은 조건의 SELECT로 대상을 확인하는 습관이 필요하다.

START TRANSACTION은 여러 변경을 한 묶음으로 다루기 시작한다. 이 실습에서 ROLLBACK은 아직 확정하지 않은 변경을 취소한다. COMMIT으로 확정한 변경이나 모든 종류의 명령을 되돌릴 수 있는 것은 아니다. 자동 번호의 다음 값은 커질 수 있지만 원래 28건의 내용은 유지된다.

이 실습은 01번 파일의 InnoDB 표를 사용한다. 사용자가 원래 보유한 실제 자료에서는 연습하지 않는다. 03번 SQL 파일에는 단계별 조회와 마지막 확인 명령까지 포함되어 있다.

8. 결과 검증과 전공과의 연결

8-1. 실제 MySQL 실행 결과 확인

별도 Linux 실습 환경의 MySQL 8.0.46에서 제공한 SQL 파일을 실행하였다. 수업용 개인 PC에서 실행한 결과로 표현하지 않았으며, 그 환경에서는 별도 재실행이 필요하다. 기준 시점은 2026년 10월 13일이다.

확인한 항목	결과
전체 기록 수	28건 확인
10월 기록만 선택	24건, 합계 195,700원
항목별 합계의 합	10월 합계와 일치
큰 지출 상위 5건	금액 내림차순·번호 오름차순 일치
HAVING 조건	식비와 쇼핑만 조회
월별 구분	9월 2건 / 10월 24건 / 11월 2건
날짜 경계	10월 1일·31일 포함, 9월 30일·11월 1일 제외
수정·삭제 연습 후	원래 28건의 값 유지
조회 재실행	결과 동일

SQL 계산값은 같은 가상 데이터를 별도로 계산한 값과도 비교하였다. 총 11개 점검을 모두 통과했으며, 원본 출력과 세부 점검 목록은 함께 제공한 실습자료 폴더에 저장하였다. 조회 속도나 여러 사용자의 동시 사용 성능은 검사하지 않았다.

8-2. 디지털보안학과와 연결되는 점

지출 내역에는 방문 장소나 생활 습관이 드러날 수 있다. 발표와 실습에는 실제 카드 내역, 계좌번호, 연락처를 넣지 않고 가상 자료만 사용하였다. 실제 서비스를 만든다면 필요한 정보만 수집하고, 허용된 사람만 조회하도록 접근 권한을 설정해야 한다.

현재 결과물은 SQL 학습용이다. 로그인, 사용자별 접근 제한, 자동 백업, 암호화 기능을 구현한 것은 아니다. 보안의 필요성을 이해하는 출발점으로 보고, 실제 사용 전에 추가 설계가 필요하다고 판단한다.

9. 실행 방법과 결론

9-1. 제공 파일 실행 순서

순서	파일	설명
1	01_준비와_데이터입력.sql	새 저장 공간·표 생성, 가상 기록 28건 입력
2	02_지출조회와_집계.sql	조회·합계·정렬 결과 확인, 반복 실행 가능
3 (선택)	03_수정삭제_선택실습.sql	연습 행으로 수정·삭제 후 되돌리기

MySQL에 접속한 후 먼저 SELECT VERSION();으로 버전을 확인한다. 명령줄에서는 SQL 파일이 있는 폴더에서 아래처럼 실행할 수 있다. root는 접속 계정 예시이며, 본인이 사용하는 계정명으로 바꾼다. 비밀번호는 -p의 입력 요청에서 입력하고 파일에 적지 않는다.

```
mysql -u root -p --default-character-set=utf8mb4 < 01_준비와_데이터입력.sql
mysql -u root -p --default-character-set=utf8mb4 < 02_지출조회와_집계.sql
```

MySQL Workbench를 이미 사용하고 있다면 SQL 파일을 열어 위 순서대로 실행해도 된다. 03번 파일은 전체 명령을 같은 연결에서 실행한다. 새 프로그램 설치는 필수가 아니다.

오류가 나면 확인할 점

상황	확인할 내용
같은 데이터베이스가 이미 존재	01번을 재실행하지 않는다. 기존 결과 확인은 02번 사용
권한 오류	접속 계정에 연습용 데이터베이스·표 생성 권한이 있는지 확인
한글이 깨짐	파일을 UTF-8로 열고 SET NAMES utf8mb4 실행 여부 확인
결과 숫자가 문서와 다름	전체 28건·10월 24건인지 확인, 예시를 중복 입력했는지 점검

9-2. 결론과 개선 방향

이번 실습은 지출 정보를 저장하는 데서 시작하여 날짜 조건, 합계, 항목별 묶기, 정렬로 확장하였다. SQL을 사용하면 같은 기록에서 서로 다른 질문에 대한 답을 구할 수 있음을 확인하였다. 핵심은 조건에 맞는 기록을 정확히 고른 뒤 계산하는 것이다.

앞으로는 지출 항목을 정해진 목록에서 선택하게 하여 표현 차이를 줄이고, 예산과 실제 지출을 비교하는 기능을 추가할 수 있다. 사용자가 직접 입력하는 화면과 사용자별 접근 권한도 후속 과제로 생각해 볼 수 있다.

10. 참고 자료와 실습 증빙

직접 확인한 자료

[1] 정성재, 「2026_02_데이터베이스_OT문서.pdf」. 수업 목표: 3–4쪽, 발표 방향과 주제: 23·29–30쪽, 발표·보고서 작성 안내: 31–33쪽.

[2] 본 과제의 실습 SQL과 MySQL 8.0.46 실행 결과. 2026년 10월 13일 별도 환경에서 확인. 실습자료 폴더의 「실제실행결과_조회.txt」, 「실제실행결과_수정삭제.txt」, 「검증결과.json」에 수록.

교수 제공 추가 학습 영상

아래 제목·링크·시간은 교수 제공 안내를 정리한 것이다. 영상·자막에 직접 접근하지 못했으므로 특정 장면이나 설명을 인용하지 않았다. 괄호의 시간은 제공된 길이 표기이며 인용 구간을 뜻하지 않는다.

[3] 데이터베이스 기초 (1/2) (26:25)

https://youtu.be/jpWXxKSMQ3s

[4] 데이터베이스 기초 (2/2) (14:13)

https://youtu.be/syMW8AVqojk

[5] create, insert 문 (21:21)

https://youtu.be/4RWdlmuGClQ

[6] select 문 (1/2) (26:05)

https://youtu.be/J8bvH4RkvFw

[7] select 문 (2/2) (30:39)

https://youtu.be/IAQQIOPosq0

[8] delete, update, show, use, alter 문 (26:42)

https://youtu.be/XM5WUSr7eXA

[9] drop, describe, load data, replace, create index, drop index 문 (31:21)

https://youtu.be/fwu3kOSUcHw

[10] grant, revoke, truncate 문 (25:32)

https://youtu.be/Vif0rAcWEaU

[11] select문에서 GROUP BY와 HAVING (12:58)

https://youtu.be/zDhDOUfNi90

주제 연결: 표 만들기·입력은 [5], 조회·정렬은 [6]–[7], 수정·삭제는 [8], 묶어서 계산하기는 [11]의 학습 주제와 연결된다. [9]–[10]의 여러 기능은 이번 핵심 실습 범위 밖의 확장 학습 자료이다.